Skill - 24

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano signals_job_recovery.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc signals_job_recovery.c -o signals_job_recovery
himasree@LAPTOP-41COT5AA:~/OSLab$ ./signals_job_recovery
========================================
     SIGNAL & JOB RECOVERY DEMONSTRATOR
========================================

----------------------------------------
1. Start Background Job
2. List Jobs
3. Stop Job (SIGSTOP)
4. Resume Job (SIGCONT)
5. Show Signal Information
6. Exit
----------------------------------------
Enter choice: 1
Enter execution time: 20

[1] Background job started.
PID  = 681
PGID = 681

----------------------------------------

Child PID: 681
1. Start Background Job
2. List Jobs
Process Group ID: 681
3. Stop Job (SIGSTOP)
```

```
2. List Jobs
Process Group ID: 681
3. Stop Job (SIGSTOP)
Background job running for 20 seconds...
4. Resume Job (SIGCONT)
Working... 1/20
5. Show Signal Information
6. Exit
----------------------------------------
Enter choice: Working... 2/20
Working... 3/20
Working... 4/20
Working... 5/20
Working... 6/20
Working... 7/20
Working... 8/20
Working... 9/20
Working... 10/20
Working... 11/20
Working... 12/20
Working... 13/20
Working... 14/20
Working... 15/20
Working... 16/20
Working... 17/20
Working... 18/20
Working... 19/20
Working... 20/20
Background job completed.
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat signals_job_recovery.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef enum
{
    RUNNING,
    STOPPED,
    DONE
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;
```

```
/* ========================================
   SIGNAL NAME
   ======================================== */

const char *signal_name(int sig)
{
    switch (sig)
    {
        case SIGINT:
            return "SIGINT";

        case SIGTSTP:
            return "SIGTSTP";

        case SIGCONT:
            return "SIGCONT";

        case SIGTERM:
            return "SIGTERM";

        default:
            return "UNKNOWN";
    }
}


/* ========================================
   STATE NAME
```

```
/* ========================================
   ADD JOB
   ======================================== */

void add_job(pid_t pid, pid_t pgid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table full.\n");
        return;
    }

    job_table[job_count].job_id = next_job_id++;
    job_table[job_count].pid = pid;
    job_table[job_count].pgid = pgid;
    job_table[job_count].state = RUNNING;

    snprintf(job_table[job_count].command,
             sizeof(job_table[job_count].command),
             "%s",
             command);

    job_count++;
}


/* ========================================
```

```
                scanf("%d", &job_id);

                resume_job(job_id);

                break;


            case 5:

                demonstrate_signals();

                break;


            case 6:

                printf("\nExiting.\n");

                return 0;


            default:

                printf("Invalid choice.\n");
        }
    }

    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$
```